SAGA PATTERN

in Spring Boot Microservices

Complete Guide with Code Examples

1. What is the Saga Pattern?

The **Saga Pattern** is a design pattern for managing **distributed transactions** across multiple microservices. Instead of a single ACID transaction spanning many services (which is impractical in a distributed system), a Saga breaks the transaction into a sequence of local transactions, each updating its own service's database and publishing an event or message.

If one step fails, the Saga executes **compensating transactions** to undo the changes made by the preceding steps — maintaining data consistency without two-phase commit (2PC).

---

2. When to Use the Saga Pattern

■ USE Saga When	■ AVOID Saga When
Multiple microservices must update their own databases atomically	All data lives in a single database (use regular ACID transactions)
You need eventual consistency across service boundaries	You need strong immediate consistency at all times
Services communicate via events or messages (async)	Low latency, synchronous workflows are required
Long-running business processes (orders, payments, shipping)	Simple CRUD operations on a single service
You want to avoid distributed two-phase commit (2PC)	Team lacks experience with eventual consistency

---

3. Two Types of Saga

3.1 Choreography-based Saga

Each service listens for events and decides what to do next. There is **no central coordinator**. Services communicate through domain events.

- Pros: Simple, decoupled, no single point of failure
- Cons: Hard to track overall workflow; business logic spread across services

3.2 Orchestration-based Saga

A central **Saga Orchestrator** (often implemented as a state machine) tells each participant what to do and handles responses. This is the most common approach in Spring Boot using **Axon Framework** or custom orchestrators.

- Pros: Clear workflow visibility; easier debugging and monitoring
- Cons: Orchestrator can become a bottleneck; adds complexity

---

4. Practical Example: Order Processing Saga

We'll build a classic e-commerce order saga involving three services: **Order Service**, **Inventory Service**, and **Payment Service**.

Step Flow:

Step	Action	On Failure (Compensate)
1	Order Service creates order (PENDING)	—
2	Inventory Service reserves stock	Release stock reservation
3	Payment Service charges customer	Refund payment, release stock
4	Order Service marks order CONFIRMED	Cancel order

---

5. Maven Dependencies (pom.xml)

```
<!-- Spring Boot + Kafka for event-driven Saga -->
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.kafka</groupId>
        <artifactId>spring-kafka</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>
```

```
    </dependency>
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
</dependencies>
```

---

6. Order Service

6.1 Order Entity

```
@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String productId;
    private int quantity;
    private double totalPrice;

    @Enumerated(EnumType.STRING)
    private OrderStatus status;  // PENDING, CONFIRMED, CANCELLED

    // Getters and setters omitted for brevity
}

public enum OrderStatus {
    PENDING, CONFIRMED, CANCELLED
}
```

6.2 Events Published by Order Service

```
// Event sent to Inventory Service
public class OrderCreatedEvent {
    private Long orderId;
    private String productId;
    private int quantity;
    // constructor, getters
}

// Compensating event if saga fails
public class OrderCancelledEvent {
    private Long orderId;
    private String productId;
    private int quantity;
}
```

6.3 Order Service (Orchestration)

```
@Service
public class OrderSagaOrchestrator {

    @Autowired private KafkaTemplate<String, Object> kafkaTemplate;
    @Autowired private OrderRepository orderRepository;

    // Step 1: Start the saga
    public Order startOrderSaga(CreateOrderRequest request) {
        Order order = new Order();
        order.setProductId(request.getProductId());
        order.setQuantity(request.getQuantity());
        order.setTotalPrice(request.getTotalPrice());
        order.setStatus(OrderStatus.PENDING);
        order = orderRepository.save(order);

        // Trigger next step: reserve inventory
        kafkaTemplate.send("reserve-inventory",
            new OrderCreatedEvent(order.getId(),
                                  order.getProductId(),
                                  order.getQuantity()));
        return order;
    }

    // Step 3 success: inventory reserved -> charge payment
    @KafkaListener(topics = "inventory-reserved")
    public void onInventoryReserved(InventoryReservedEvent event) {
        kafkaTemplate.send("process-payment",
            new ProcessPaymentEvent(event.getOrderId(),
                                    getOrderTotal(event.getOrderId())));
    }

    // Step 3 fail: inventory failed -> cancel order
    @KafkaListener(topics = "inventory-failed")
    public void onInventoryFailed(InventoryFailedEvent event) {
        cancelOrder(event.getOrderId(), "Inventory unavailable");
    }

    // Step 4 success: payment done -> confirm order
    @KafkaListener(topics = "payment-processed")
    public void onPaymentProcessed(PaymentProcessedEvent event) {
        orderRepository.findById(event.getOrderId()).ifPresent(order -> {
            order.setStatus(OrderStatus.CONFIRMED);
            orderRepository.save(order);
        });
    }

    // Step 4 fail: payment failed -> compensate (release stock + cancel)
    @KafkaListener(topics = "payment-failed")
    public void onPaymentFailed(PaymentFailedEvent event) {
```

```
        // Compensating transaction: release reserved inventory
        kafkaTemplate.send("release-inventory",
            new OrderCancelledEvent(event.getOrderId(),
                                    getProductId(event.getOrderId()),
                                    getQuantity(event.getOrderId())));
        cancelOrder(event.getOrderId(), "Payment failed");
    }

    private void cancelOrder(Long orderId, String reason) {
        orderRepository.findById(orderId).ifPresent(order -> {
            order.setStatus(OrderStatus.CANCELLED);
            orderRepository.save(order);
        });
    }
}
```

---

7. Inventory Service

```
@Service
public class InventoryService {

    @Autowired private InventoryRepository inventoryRepository;
    @Autowired private KafkaTemplate<String, Object> kafkaTemplate;

    @KafkaListener(topics = "reserve-inventory")
    public void reserveInventory(OrderCreatedEvent event) {
        Inventory inventory = inventoryRepository
            .findByProductId(event.getProductId())
            .orElseThrow(() -> new RuntimeException("Product not found"));

        if (inventory.getAvailableQuantity() >= event.getQuantity()) {
            // Reserve stock (local transaction)
            inventory.setAvailableQuantity(
                inventory.getAvailableQuantity() - event.getQuantity());
            inventoryRepository.save(inventory);

            // Publish success event
            kafkaTemplate.send("inventory-reserved",
                new InventoryReservedEvent(event.getOrderId(),
                                           event.getProductId()));
        } else {
            // Publish failure event (triggers compensation)
            kafkaTemplate.send("inventory-failed",
                new InventoryFailedEvent(event.getOrderId(),
                                         "Insufficient stock"));
        }
    }

    // Compensating transaction: release reserved stock
    @KafkaListener(topics = "release-inventory")
```

```
    public void releaseInventory(OrderCancelledEvent event) {
        Inventory inventory = inventoryRepository
            .findByProductId(event.getProductId()).get();
        inventory.setAvailableQuantity(
            inventory.getAvailableQuantity() + event.getQuantity());
        inventoryRepository.save(inventory);
    }
}
```

---

8. Payment Service

```
@Service
public class PaymentService {

    @Autowired private PaymentRepository paymentRepository;
    @Autowired private KafkaTemplate<String, Object> kafkaTemplate;

    @KafkaListener(topics = "process-payment")
    public void processPayment(ProcessPaymentEvent event) {
        try {
            // Simulate payment gateway call
            boolean success = chargeCustomer(event.getOrderId(),
                                             event.getAmount());
            if (success) {
                Payment payment = new Payment(event.getOrderId(),
                                              event.getAmount(),
                                              PaymentStatus.COMPLETED);
                paymentRepository.save(payment);

                kafkaTemplate.send("payment-processed",
                    new PaymentProcessedEvent(event.getOrderId()));
            } else {
                kafkaTemplate.send("payment-failed",
                    new PaymentFailedEvent(event.getOrderId(),
                                           "Payment gateway declined"));
            }
        } catch (Exception e) {
            kafkaTemplate.send("payment-failed",
                new PaymentFailedEvent(event.getOrderId(), e.getMessage()));
        }
    }

    // Compensating transaction: refund payment
    @KafkaListener(topics = "refund-payment")
    public void refundPayment(RefundPaymentEvent event) {
        paymentRepository.findByOrderId(event.getOrderId())
            .ifPresent(payment -> {
                payment.setStatus(PaymentStatus.REFUNDED);
                paymentRepository.save(payment);
                // Call payment gateway refund API here
```

```
            });
    }

    private boolean chargeCustomer(Long orderId, double amount) {
        // Integration with payment gateway (Stripe, PayPal, etc.)
        return Math.random() > 0.1; // 90% success for demo
    }
}
```

---

9. Kafka Configuration

```
# application.yml
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
    consumer:
      group-id: saga-group
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      properties:
        spring.json.trusted.packages: "com.example.saga.events"

# Kafka Topics used in this Saga:
# reserve-inventory   -> OrderService  -> InventoryService
# inventory-reserved  -> InventoryService -> OrderService
# inventory-failed    -> InventoryService -> OrderService
# process-payment     -> OrderService  -> PaymentService
# payment-processed   -> PaymentService -> OrderService
# payment-failed      -> PaymentService -> OrderService
# release-inventory   -> OrderService  -> InventoryService (compensation)
```

---

10. Saga State & Idempotency

To handle retries and duplicate events, track the saga state in a dedicated table. Each step is idempotent — processing the same event twice produces the same result.

```
@Entity
public class SagaState {
    @Id private Long orderId;

    @Enumerated(EnumType.STRING)
    private SagaStep currentStep;

    @Enumerated(EnumType.STRING)
    private SagaStatus status;    // IN_PROGRESS, COMPLETED, COMPENSATING, FAILED

    private String failureReason;
```

```
    private LocalDateTime updatedAt;
}

public enum SagaStep {
    ORDER_CREATED,
    INVENTORY_RESERVED,
    PAYMENT_PROCESSED,
    ORDER_CONFIRMED,
    // Compensating steps
    PAYMENT_REFUNDED,
    INVENTORY_RELEASED,
    ORDER_CANCELLED
}
```

---

11. Best Practices & Key Principles

Idempotency: Every step must be idempotent. Use a unique event/correlation ID to detect and ignore duplicate messages.

Compensating Transactions: Design compensation before implementing forward steps. Each action must have a defined undo operation.

Outbox Pattern: Save events to a local 'outbox' table in the same DB transaction as the business data, then publish from there to avoid dual-write problems.

Dead Letter Queues: Configure DLQs in Kafka for unprocessable messages. Monitor and alert on DLQ growth.

Saga Timeout: Add a saga timeout mechanism. If a saga doesn't complete within X minutes, auto-compensate.

Observability: Use distributed tracing (Zipkin/Jaeger) and correlation IDs across all services to trace a saga end-to-end.

Testing: Test both happy path and all compensation paths. Use embedded Kafka for integration tests.

---

12. Saga vs Two-Phase Commit (2PC)

Aspect	Saga Pattern	Two-Phase Commit (2PC)
Consistency	Eventual consistency	Strong consistency
Coupling	Loosely coupled	Tightly coupled
Performance	High (async)	Low (blocking locks)
Availability	High	Lower (coordinator lock)
Complexity	Medium-High	Very High
Best for	Microservices	Monolith / same DB vendor

---

13. Summary

The **Saga Pattern** is essential for managing distributed transactions in Spring Boot microservices. Use it when you need data consistency across multiple services without sacrificing availability or coupling.

Key takeaways:

- Break long transactions into local steps with compensating rollbacks
- Choose **Choreography** for simple flows, **Orchestration** for complex ones
- Always implement idempotency and the Outbox Pattern
- Design compensation logic *before* forward logic
- Add distributed tracing for visibility across services